# Cash Is a File

*Forward it. The first open spends it.*

**ZCrypt**

*A Post-Quantum Shielded Ledger*
*with Whole-Pool Anonymity*

Echelon Labs LLC

Christian Papathanasiou (founder@zcrypt.network)

*November 2026 · Version 9.8*

## Abstract

ZCrypt is a proof-of-stake ledger with one shielded pool and no transparent transfers. Every unit of value, native or otherwise, lives in that pool as a hiding commitment. Coins move by one kind of shielded transfer, which publishes two nullifiers, three fresh commitments, three encrypted notes, a fee, an anchor and a proof, and nothing else. The proof is a STARK: hash functions all the way down, no trusted setup, and nothing in it that a quantum computer is expected to break. A spend proves membership in the global commitment tree, so the anonymity set is every coin ever created rather than a handful of sampled decoys, and a one-input spend is padded so that even the number of inputs is hidden. Ownership is a hash preimage proven inside the circuit; there are no signatures in the value layer. Notes are encrypted to ML-KEM-768 keys, so ciphertexts recorded today stay closed against a future quantum adversary.

Around that core the rules are transparent. Every transparent operation is a short bytecode program that consensus runs, and the shielded transfer has a bytecode specification tested against the circuit. Consensus is BFT with instant finality. Issuance follows a fixed 21-million schedule committed in the state root. Registered assets share the pool with hidden type identifiers and are issued under rotatable $M$-of-$N$ ML-DSA authorities. Bitcoin moves in and out through SPV-verified atomic swaps: a mint requires proof that BTC was locked to a secp256k1 reserve key, and that key can spend the locked coins. A payment can also be a file: a bearer coin and the gas that spends it, sent over any channel. Whoever sweeps it first owns it, so until the sweep is in a block the file is worth the giver's word. Every transfer, swap lock and redemption pays a fixed 0.01 ZCRYPT that is burned. A ZCRYPT transfer burns it inside its own proof, so a wallet holding a single coin can pay the fee, pay the recipient and keep its change in one transaction; any other asset burns it in a bundled ZCRYPT gas proof, and the asset being moved is never the fee.


## 1 Introduction

Bitcoin solved double-spending without a central party by making the ledger public [1]. That was the right call for the problem it set out to solve, and it is why anyone can audit the supply. It has a cost that Bitcoin never claimed to address: every payment is a permanent public record of who paid whom, how much, and when. Coins acquire histories, histories acquire reputations, and an exchange will refuse a coin that passed through the wrong hands. Cash has none of this, because a banknote carries no ledger.

ZCrypt is built so that the ledger records as little as a banknote does. It also has a bearer instrument that is a file (§4.3): a coin's secret and the gas to move it, handed over on any channel. Unlike a banknote, the file can be copied, but the coin cannot be double spent: it can only be swept once, and it belongs to whoever sweeps it first.

Earlier systems went part of the way. Ring signatures [2] hide a spend among a few sampled decoys, and the hiding erodes as rings overlap. Shielded pools [3] hide far more; Zcash's newest pool dropped the trusted setup in 2022 with Halo 2, the production descendant of [7], and multi-asset pools with in-pool exchange exist today [8, 9]. All of them rest on discrete-logarithm assumptions, which are precisely the assumptions a quantum computer breaks. Most also run a transparent ledger beside the private one, and since most value stays on the transparent side, the pool is smaller than it looks. Zcash Orchard proves membership against its whole note tree [4], and Monero's FCMP++ is set to replace sampled rings with full-set membership [5]. Neptune Cash is a proof-of-work chain whose transactions are hash-based STARKs over the Triton VM, with notes under lattice keys [6].

ZCrypt is a proof-of-stake ledger with one shielded pool and no transparent transfer beside it. Gas is a fixed burn of the native coin. Bitcoin enters through a light client that lives inside consensus. And a payment can be a file rather than an address. Four decisions shape everything else in this paper.

1. **One pool, one path.** There is no transparent transfer. A coin exists only as a hiding commitment in one global pool, and it moves only by a shielded spend. Nobody can opt out of privacy on a transfer, so nobody fragments the anonymity set. Bonding, reward claims, asset registration and escrows are separate, transparent operations (§7).
2. **Whole-pool anonymity.** A spend proves, inside the circuit, that the spent coin sits somewhere in the global tree of every coin ever created. That tree is the anonymity set of every spend.
3. **Post-quantum on the spend path.** The proof system is a hash-based STARK with no setup. Spend authority is a hash preimage. Note encryption is a lattice KEM. No discrete-log or pairing assumption stands between an attacker and a pool coin. The Bitcoin bridge is the exception: a secp256k1 signature picks the recipient of a zBTC mint and rotates the reserve key, and ZCrypt consensus checks those signatures itself.
4. **Transparent rules, private state.** The ledger hides who paid whom and how much. It does not hide the rules. Every transparent operation has a short bytecode program that consensus runs. The shielded transfer has a bytecode specification too, though for it consensus checks the proof system's constraints instead. Total issuance is a function of block height committed in the state root, so anyone can confirm the schedule was followed.

The limits are stated where they bind. The bootstrap sentinel is per consensus key and can be satisfied by split-

ting stake. Reserve custody for the Bitcoin bridge sits with one secp256k1 key. Zero knowledge of the proofs is unproved. The hash permutation and the FRI protocol sit outside the machine-checked model of the circuit. A transfer is about two megabytes.

§2 gives the whole system in a page. §3 to §5 cover the cryptography, the coin and the shielded transfer; §6 to §8 the auditability VM, consensus and money; §9 to §11 other assets, the Bitcoin bridge and in-pool swaps. §12 describes the wallet, §13 and §14 say what an observer sees and what an attacker would need, and §15 sketches two deployments.

# 2 System Overview

ZCrypt is a replicated state machine run by a proof-of-stake validator set with instant finality (§7). The state it commits to is small, and its shape is entirely public: an append-only Merkle tree of coin commitments, one leaf per coin ever created; a sparse Merkle tree of nullifiers, one entry per coin ever spent; the staking ledger, the asset registry and an escrow book for swaps; the Bitcoin header chain and the queue of immature zBTC mints (§10); the bootstrap sentinel record (§7.4); and the window of recent anchor roots (§5). Hashing those nine roots together gives the application state root that consensus agrees on every block.

The wallet does the heavy lifting. To pay, it picks its own coins, builds a zero-knowledge proof locally, and broadcasts a transaction containing the proof, its public inputs, and nothing else. Validators check the proof, check that the nullifiers are fresh, append the new commitments, and never learn an amount, an asset type or a participant. To receive, a wallet trial-decrypts the encrypted notes attached to on-chain transactions; whichever notes open under its key are payments to it.

Everything that moves value, whether a payment, a gas burn, a stake bond, a bridge deposit or redemption, or a reward claim, ends up as coins entering or leaving this one pool.

# 3 Cryptographic Foundations

Three primitives carry the design: an algebraic hash that is cheap inside a proof, a hash-based proof system, and a lattice encryption scheme for notes.

## 3.1 Field and Hash: Poseidon2 over BabyBear

Every object the circuit touches, whether a commitment, a nullifier, a public key or a Merkle node, is computed with one hash: the Poseidon2 permutation [10] over the BabyBear prime field, $p = 15 \cdot 2^{27} + 1 = 2013265921$. The permutation has width 16, an $x^7$ S-box, 8 full rounds and 13 partial rounds. The 16-element input is permuted once and the digest is the first 8 lanes of $P(x)+x$, written $\mathrm{Trunc}_8(P(x)+x)$. In the outer block of a commitment, a nullifier or a public key, a constant role tag occupies the last lane, so those three objects are different inputs even when the rest of the lanes agree. Merkle nodes use the same compression with one child in lanes 0–7 and the other in lanes 8–15.

The digest is 8 limbs, about 247 bits of range. In the ideal-permutation model a classical preimage search costs about $2^{247}$ permutations and a generic collision search about $2^{124}$. Against a quantum attacker, Grover's algorithm puts a preimage at about $2^{124}$, and the Brassard–Høyer–Tapp algorithm gives collisions at about $2^{82}$ at the cost of quantum memory of the same order, so under most cost models $2^{124}$ remains the realistic bound. I therefore claim roughly 124-bit classical collision resistance and 124-bit quantum preimage resistance, and not 128-bit post-quantum collision resistance. These are figures for the compression in an ideal model, not a proved lower bound on Poseidon2 itself. The implementation reports every figure in this section from its compiled constants (zcrypt –proof-params).

An algebraic hash is used because the same function has to be cheap both natively and inside the proof. One Poseidon2 permutation costs 298 trace columns in the circuit; SHA-256 would cost tens of thousands.

Outside the circuit, byte-level hashing uses SHA3-256 [11] for state roots, transaction binding, key derivation and escrow identifiers. Every such hash starts with a length-prefixed ASCII domain tag and length-prefixes each field, so two different structures never hash to the same bytes. The one exception is the chain-domain tag inside the proof's public inputs, which uses a raw eight-byte tag. The FRI commitment layer inside the proof system uses Keccak. Nowhere in the value layer does anything depend on discrete logarithms being hard.

## 3.2 The Proof System: a Hiding STARK

Transfer proofs are STARKs [12] in the AIR/FRI style [13], produced and verified with Plonky3 release 0.6.1 [15]. The version is pinned in the lockfile and a test checks it against the version cited here. The arithmetization is a two-dimensional execution trace over BabyBear with constraints enforced on every row.

The parameters are the same in every build; there is no lower-security variant. FRI runs with a blowup factor of 4 (rate $2^{-2}$), folding arity up to 4 and a final polynomial of length 2. There are 112 FRI queries, with 20 bits of proof-of-work grinding on both the commitment phase and the query phase. Fiat–Shamir challenges come from a degree-5 extension of BabyBear, about $2^{155}$ elements; a degree-4 extension would put the out-of-domain terms under 128 bits.

Two soundness figures are reported, and they differ. The conjectured list-decoding figure is $2 \times 112 + 20 = 244$ bits, the same formula Plonky3 reports, which assumes proximity gaps up to the capacity of the code. The up-to-capacity form of that conjecture is false [14]. The Johnson bound, which current analyses treat as

proven, gives about one bit per query at this rate: $(2-1)\times 112+20=132$ bits, before degree and batching losses. Those losses sit under the degree-5 extension and do not restore 244. The prover asserts at configuration time that the Johnson figure is at least 100 bits, and a node refuses to start on a real-value network below 128; both floors check the Johnson figure, and 132 passes both. The arithmetic of both figures is machine-checked in the Lean development of §14. The per-query rejection probability itself is the proximity-gap theorem, and that is not.

Hiding is a different matter. Zero knowledge comes from the polynomial commitment layer, not from the AIR. Plonky3's hiding mode doubles the trace: the production trace is 128 rows, one random row is interleaved with each of them, four random codewords blind the quotient, and every Merkle leaf carries a salt from a ChaCha20 generator seeded by the operating system. Two proofs of the same statement differ in their bytes, which shows the seed is fresh; it does not show the witness is hidden. The 112 queries plus two out-of-domain openings come to 122 openings against 128 random rows, and that count leaves out the sibling values each folding round reveals. No simulation argument exists that what remains is enough. If this layer leaks, every proof already published gives up its witness to anyone who kept a copy, and a later upgrade does not take those proofs back. The 112 queries are also why each proof is about two megabytes. There is no trusted setup; the only preprocessing is a public matrix of selector columns that prover and verifier both derive from the circuit.

### 3.3 Note Encryption: ML-KEM-768

A payment has to get the coin's opening (value, blind, asset identifier) to the recipient and to nobody else. Each output slot carries an encrypted note. The sender encapsulates to the recipient's ML-KEM-768 [16] public key, expands the shared secret with HKDF-SHA256 [17] with the KEM ciphertext and the recipient's encapsulation key bound into the derivation, and encrypts under ChaCha20-Poly1305 [18]. The plaintext is 80 bytes (value, the eight-limb blind, asset) padded to a fixed 256, so every note on chain is the same 1,380-byte wire object whatever it holds.

Recipients scan by trial decryption: a note either authenticates under the wallet's decapsulation key or it is noise. ML-KEM is a lattice scheme standardized for post-quantum confidentiality, so notes harvested from the chain today cannot be opened later by a quantum computer.

An output slot that should not carry a real note, for instance when the payment is delivered out of band (§4.3), carries a cover note: a correctly shaped ciphertext encapsulated to a key that is then thrown away. Nobody watching the chain can tell the two apart.

## 4 Coins, Keys, and Addresses

### 4.1 The Coin

A coin is four numbers: a value $v$ of up to 52 bits in atomic units; a blinding factor $b$ of eight field limbs, about 248 bits, so recovering the blind of a guessed amount is a $2^{248}$ search classically and about $2^{124}$ under Grover; a spend secret $s$ of eight field limbs; and an asset identifier $a$ that fits in one BabyBear lane, about 31 bits. ZCRYPT is identifier 0. Other assets get the reduced hash of their ticker (§9).

The commitment is the only thing derived from the coin that appears on chain when it is created:

$$C=\mathsf{P2}\big(v_{\mathrm{lo}},v_{\mathrm{hi}},\,\mathsf{P2}(pk,b),\,a\big),\qquad pk=\mathsf{P2}(s),$$

with $\mathsf{P2}$ the compression of §3.1 and the value split into two 26-bit limbs. The inner compression absorbs the eight-limb public key and the eight-limb blind, which exactly fill one sixteen-lane block; the outer compression binds that digest to the value, the asset and the commitment role tag. Because the recipient's public key is inside the hash, the creator of a coin names its owner. Opening the same commitment two ways is a collision of the compression.

The nullifier appears when the coin is spent:

$$nf=\mathsf{P2}\big(\mathsf{P2}(s,\mathsf{P2}(pk,b)),\,v_{\mathrm{lo}},v_{\mathrm{hi}},\,a\big).$$

Its inner block absorbs the spend secret beside the commitment's own inner digest rather than beside the raw blind, and the reason is worth recording. The sender of a coin chooses its blind. When the blind sat directly in the nullifier's inner block, the blind $(0,\dots,0,\mathrm{tag}_{pk})$ turned that block into the public-key block and let a sender compute the recipient's nullifier. Absorbing the digest leaves no such lane.

The same opening always produces the same nullifier, and producing it requires $s$, so spending one coin twice collides in the public nullifier set and the second spend is rejected. The leaf position is not an input. Zcash's "Faerie Gold" attack, in which two notes are built to share one nullifier, was closed by making the nullifier seed unique per transaction in Sprout and by binding it to the note's tree position in Sapling [3]. This nullifier has no free lane for a position, so ZCrypt closes the equivalent attack at the tree: consensus refuses a commitment that is already a leaf, including two copies in one transaction, and a snapshot with repeated leaves does not load. Nobody without the spend secret can connect a nullifier to its commitment.

Values are capped at $2^{52}-1$ atomic units per coin. With six decimals that is about 4.5 billion ZCRYPT, far more than will ever exist, so the cap never binds.

### 4.2 Key Hierarchy

An account comes from a 24-word BIP-39 mnemonic [19] carrying 256 bits of entropy. The seed is domain-hashed

into the spend secret $s$ and into a deterministic ML-KEM-768 keypair for notes, so both come back from the phrase alone. The spend public key $pk = \mathsf{P2}(s)$ is a hash image, not a group element.

### 4.3 Addresses and Payment Instruments

An address is a kind tag, the 64-byte spend public key and the 1,184-byte ML-KEM encapsulation key, encoded in bech32m [20] with human-readable part zc. The payload is 1,249 bytes and the string is 2,008 characters, far past the 90-character limit that BIP-350 inherits from BIP-173, so the encoder is a ZCrypt-specific long form: the same character set and checksum polynomial with the length limit removed. Within the 90-character limit the checksum detects up to four substitutions, and the code is designed for strings up to 1,023 characters; at 2,008 characters that guarantee becomes probabilistic, and a random corruption passes with probability $2^{-30}$. The decoder checks the exact payload length, rejects any limb at or above the field order, and rejects an ML-KEM key that does not re-encode to itself, so an address has exactly one byte string and one encoded string.

Three payment instruments share this machinery. A *normal address* is the account's primary address; payments to it carry a real encrypted note on chain, are found by scanning, and are recoverable from the mnemonic.

A *stealth address* has the same shape, backed by an independent key set. Stealth sub-accounts derived from the master seed and an index are what the wallet uses for change, so change cannot be linked to the published address. Stealth receive addresses that a user creates on purpose use fresh random seeds; they are not derivable from the mnemonic and the account's view key cannot see them. For a stealth payment the on-chain slot carries a cover note and the real note travels out of band, as a string beginning zct1. On the wire a stealth address looks exactly like a normal one.

A *bearer coin* is handed over as a file, and the file is the only handoff. Its text is a header line zcrypt-bearer/1, a coin line and a gas line. The coin line is the value, in any asset. The gas line is a ZCRYPT bearer coin of 0.01, the gas the claim burns. A lone zcb1 string is not a payment. Each token is 144 bytes (value, blind, spend secret, asset; each limb an 8-byte little-endian canonical element) in the long-form encoding with prefix zcb. Creating the file costs the sender 0.03 ZCRYPT on top of the value: the gas coin that travels inside the file, and 0.01 of gas for each of the two mints. Claiming sweeps the value to a fresh coin under the claimant's key and pays the gas from the file, so the claimer needs no prior balance. The first sweep wins. A copy kept by the sender, or by anyone else, can still spend the coin until that sweep is in a block. The mnemonic does not restore a bearer.

An account can also hand out its ML-KEM key pair without the spend secret, as a *view key*. Whoever holds it can decrypt every incoming note to the normal address, amounts and asset types included, but cannot spend and cannot derive nullifiers, so cannot see when the funds leave. Stealth and bearer instruments are outside the view key's scope by construction. This is how an account discloses selectively, to an auditor say, without giving up custody.

### 4.4 No Signatures

There is no signature scheme in the value layer. Authority to spend is knowledge of $s$, and the circuit is where that knowledge is proven: the spent commitment is constrained to open to $pk = \mathsf{P2}(s)$ for the $s$ the prover supplies, and the published nullifier is constrained to derive from that same $s$.

Malleability is closed separately. Every ciphertext, commitment and fee, and the proof itself, are bound into a transaction-binding hash that is stored next to each nullifier of that proof, so those fields cannot be swapped after broadcast. A second proof packaged beside the first is bound by its own auxiliary digest, and the gas proof of §5.7 commits to the transaction it pays for. No ECDSA or EdDSA stands between the spend secret and an attacker; the bridge signatures of §10 are outside that sentence.

It also means there is no in-protocol multisig for user funds. Threshold custody of a pool coin means splitting $s$, or the mnemonic behind it, by secret sharing outside the protocol. Issuance authorities are threshold-governed on chain (§9.1); individual coins are not.

## 5 The Shielded Transfer

### 5.1 Transaction Contents

A transfer publishes two nullifiers (one per input slot), three output commitments, three fixed-size encrypted notes (real or cover), a fee field, an anchor root, and the STARK proof. An output slot a payment does not need holds a value-zero commitment to the burn key under a fresh blind, so the count never varies.

Hidden: amounts, sender, recipient, which coins were spent, and how many (§5.6). Public: that some valid spend of pool coins happened, the gas burn, and the transaction's tag, which says whether the transfer moved ZCRYPT or some other asset (§5.7).

### 5.2 The Proven Statement

The proof has 62 public field elements: the three output commitments (8 limbs each), the fee, the two nullifiers, the anchor root, a five-limb chain-domain tag, and an eight-limb auxiliary digest. The digest binds the transaction's encrypted notes into the proof; for escrow and staking transactions it binds their plaintext parameters as well; for a bundled gas proof it binds a hash of the transaction being paid for (§5.7).

Against those inputs the proof says that the prover knows openings $(v_i, b_i, s_i, a)$ for each input and $(v'_j, b'_j, pk'_j, a)$ for each output such that:

1. **Membership.** Each active input's commitment is a leaf of the depth-62 commitment tree under the public anchor root, shown by an in-circuit Merkle path of Poseidon2 compressions. The path proves existence and position and reveals neither.
2. **Ownership.** Each input commitment opens to $pk_i = \mathsf{P2}(s_i)$; the prover knows the spend secret.
3. **Nullifier correctness.** Each published nullifier is the nullifier of §4 for the same witness values, so it cannot be forged or detached from its coin.
4. **Output correctness.** Each public output commitment opens to its witness values with a well-formed layout.
5. **Range.** Every value is decomposed into 52 constrained bits and the fee into 26, so nothing can overflow the field.
6. **Conservation.** $\sum_i v_i = \sum_j v'_j + fee$, enforced limb by limb with a range-checked carry, so it holds over the integers and not just modulo $p$.
7. **Single asset.** Inputs and outputs all carry one asset identifier, so value cannot leak from one asset into another.
8. **Domain.** A hash of the chain identifier is a public input, so a proof made for one network is useless on another.

The membership proof is the centre of the privacy model. A spend hides among every leaf of the tree, every coin ever created, and the set grows with the system. Since nothing is sampled, an intersection attack has nothing to intersect.

## 5.3 Anchors and the Root Window

The tree is append-only, so its root changes in every block that adds coins. A proof is built against whatever root the wallet last synced. Consensus accepts any anchor that is a block-boundary root from the last 1,000 blocks or one of the 64 most recent intra-block roots, and checks the anchor before verifying the proof. The window is itself part of the state root, so validators cannot disagree about which anchors are live. Wallets never race the chain, and a proof built against ancient state simply expires.

## 5.4 Tree Capacity

The circuit fixes the Merkle depth at 62, so the tree holds $2^{62}$ leaves, about $4.6 \times 10^{18}$. Every output commitment takes a leaf, three per proof, so a native ZCRYPT transfer adds three leaves and a bundled asset transfer six, and the tree holds about $1.5 \times 10^{18}$ transfers. At a sustained million transfers per second it fills in roughly fifty thousand years. There is no rollover mechanism and none is planned: the bound is real but not reachable, and the ledger keeps a single whole-history anonymity set for its lifetime.

The depth costs nothing in the production circuit. The tall trace (§3.2) is 128 rows: two for the inner and core constraints, one per level per input for the authentication paths, so two inputs at depth 62 occupy rows 2 to 125 and two rows remain padding. The prover evaluates every row whether it is a path row or padding, so proof size, proving time and verification cost are those of the 128-row trace at any depth up to 63. What grows with depth is the private witness, the per-leaf append cost on the node, and the wide reference circuit used in testing. An earlier parameter of depth 32 would have filled in months at payment-network throughput and would have needed a generation rollover with a per-generation anonymity set. Raising the depth before genesis removed both.

## 5.5 The Nullifier Set

Spent-coin state is a sparse Merkle tree of height 256 keyed by a hash of the nullifier. The decoder rejects any limb that is not in canonical field form, so no second encoding of the same nullifier exists. The leaf value is the spending transaction's binding hash, so two conflicting spends of one coin are distinguishable in the tree itself. Insertion is first-spend-wins, and a transaction with any nullifier already present is rejected. The all-zero nullifier is reserved and rejected. The tree gives compact inclusion and non-inclusion proofs, which nodes serve on request; the reference wallet answers "is this coin spent?" from the blocks it already stores instead (§12).

## 5.6 Arity Hiding

A transaction always publishes exactly two nullifiers. When a wallet spends one coin, the second input slot gets a *phantom* coin of value zero. Its spend secret and blind are derived by folding a domain constant and the slot index, through Poseidon2, with each real input's secret coin tag. That tag hashes the real coin's spend secret, value, blind and asset under a domain distinct from the nullifier, and it is never published. The phantom is exempt from tree membership, since a value-zero input cannot create value, but it publishes a real, unpredictable nullifier like any other input, and consensus records that nullifier as spent. One-input and two-input spends therefore look identical on chain.

Because the tag binds the whole coin, distinct coin openings produce distinct phantoms even when a spend secret is reused. Deriving the phantom from the published nullifier instead would let an observer recompute the padding slot and see that the spend was short; the secret tag is what keeps that hidden.

## 5.7 Gas

Every transfer pays a fixed 0.01 ZCRYPT, and the fee is burned; nobody receives it. Since the pool hides amounts, the fee cannot be a percentage. It is paid as a revealed burn: consensus requires the first output of a

gas-paying proof to be the canonical burn commitment

$$C_{\text{gas}} = \mathsf{P2}(v_{\text{gas}},\, b_{\text{gas}},\, pk_{\text{burn}},\, 0),$$

with the value and the blind both revealed in the transaction so anyone can recompute it. The burn key $pk_{\text{burn}}$ is a fixed constant with no known Poseidon2 preimage; coins committed to it are unspendable and gone for good.

A native ZCRYPT transfer is a single proof whose first output is $C_{\text{gas}}$ and whose other two outputs are the payment and the change. A proof is single-asset and the revealed burn is a ZCRYPT coin, so the hidden inputs and outputs are ZCRYPT too. The in-transaction fee field is zero and no second coin is involved. This is the shape that keeps a wallet from being stranded: a single coin, however it arrived, pays the fee, pays the recipient and keeps its change. The wallet normally burns exactly 0.01; the rule is an inequality so that dust below the fee can be folded into a later coin's burn instead of surviving as unspendable change.

A transfer of any other asset cannot carry a ZCRYPT burn inside its own proof, so it is two proofs applied atomically: the main transaction, with a zero fee field, and a gas transaction spending ZCRYPT whose first output is $C_{\text{gas}}$ at exactly 0.01. The two proofs may not share a coin, and both must verify before either takes effect. The gas proof's auxiliary digest commits to a domain-separated hash of the main proof's binding hash, so a gas proof does not verify as payment for a different transaction. Registration, escrow opens and escrow locks of a non-ZCRYPT asset carry the same gas note. ZCRYPT is in this sense the universal gas asset: moving any asset consumes native coin, and the asset being moved is never charged.

The two shapes are not indistinguishable. They carry different transaction tags, and the native form shows its burn value and blind in the clear, so an observer can tell a ZCRYPT transfer from a transfer of some other asset. Within the bundled form the asset stays hidden, so all registered assets share one anonymity set among themselves and ZCRYPT transfers share another. §13 returns to this.

Why burn, and why fix the amount? Paying the fee to validators would mean crediting a hidden-amount transaction to a public ledger, which either reveals the fee or adds a second accounting path. A fixed burn needs neither. A fee market would also cost privacy: whatever fee a user picks is a public number on an otherwise uniform transaction, and wallets that pick differently become fingerprintable. The 0.01 figure is a consensus parameter that only a coordinated upgrade can change.

The constant has a cost of its own. A valid flood cannot be outbid, because every honest transaction pays the same fee; the sender who can prove fastest fills the block. On the laptop in §6.2 a transfer takes between about 1.2 s and 2.5 s to prove, so one machine produces well under one transfer a second and burns 0.01 ZCRYPT on each. An invalid proof is cheaper: it costs a validator about 23 ms of verification and the bandwidth of a megabyte-scale blob, and burns nothing. Proposers earn none of the fee, so declining to include a transaction costs them nothing. While emission is large, staking rewards pay validators to be online; as the schedule halves away, that payment shrinks and the burned fee does not replace it.

# 6 Transparent Rules: the Auditability VM

A privacy system asks people to trust rules they cannot watch run. ZCrypt narrows that trust to something a person can read: every transaction type has a short bytecode program for a small VM built for the purpose.

For the shielded transfer the program is seven opcodes:

CHECK_RANGE · CHECK_MEMBERSHIP · CHECK_SINGLE_ASSET · CHECK_CONSERVATION · COMPUTE_NULLIFIERS · COMPUTE_OUTPUTS · HALT

Each opcode implements one constraint family of the circuit in plain integer arithmetic over the clear witness, down to reproducing the phantom-coin padding bit for bit. This program is the specification of the transfer. Consensus cannot run it, because the chain never sees a witness, so it runs where witnesses exist: a differential test suite asserts VM ≡ circuit on valid and adversarial witnesses, and every wallet runs the VM after proving and refuses to broadcast if the VM and the proof disagree. Should the prover ever be wrong, the wallet fails closed rather than sending. What consensus enforces for a transfer is the circuit's constraint system, the AIR, and §14 says what the machine-checked model of that AIR covers.

For transparent operations (staking, claims, registration, authority rotation, escrows) a second instruction set checks stake minimums, claimable balances, ticker validity, escrow state, timeouts, hashlock preimages, burn outputs, threshold signatures against a registered authority, and, for a swap lock placed without a gas note, that its gas was paid (§11). Here the VM is the gate: consensus runs the program and its answer is the accept or reject decision. Each program is two to four bytes, a few dozen bytes in all, and an auditor can read them and know what those operations are allowed to do.

## 6.1 Circuit Engineering

The transfer circuit exists in two differentially tested forms that prove the same statement with the same public inputs. The wide reference trace is 32 rows by 43,128 columns, all 140 Poseidon2 permutations side by side. The production tall trace is 128 rows by 2,993 columns plus 7 preprocessed selector columns, and it runs the 124 Merkle-path compressions one per row. Proof size in a STARK grows linearly with width and only logarithmically with height, which is why the tall layout produces

much smaller proofs. The wide circuit is kept as the reference the tall one is checked against; cross-verification between the two, and against the VM, is part of the test suite.

### 6.2 Measured Performance

Medians on an Apple M3 Max laptop (14 cores, 36 GB), release build, 12 proving runs, 40 verification runs and 2,000 VM runs per configuration:

| | **2-in/3-out** | **1-in/3-out** |
|---|---|---|
| Prove (build the tx) | 1.31 s | 1.19 s |
| Verify (network check) | 22.7 ms | 22.7 ms |
| Transparent VM audit | 83.3 $\mu$s | 54.8 $\mu$s |
| Proof size | 2104 KB | 2105 KB |
| Verify throughput / core | ∼44/s | ∼44/s |

The identical proof sizes are arity hiding showing up in the numbers: a one-input spend is byte for byte the same shape as a two-input spend. Proving time varied between roughly 0.6 s and 3 s across runs; the medians are quoted. A native ZCRYPT transfer is one proof, so a wallet builds a complete private payment in about 1.2 s and a validator core admits it in about 23 ms, near 44 transfers a second per core. A bundled asset transfer is two proofs: about 2.5 s to build and 45 ms to admit.

Bandwidth binds well below that rate. A native transfer carries about 2 MB of proof and a bundled transfer about 4 MB. The reference image raises CometBFT's per-transaction mempool cap to 5 MB; the block cap is CometBFT's stock value of about 21 MB, so a block holds about ten native transfers or five bundled ones. At ten native transfers a second each validator moves about 170 Mbit/s of proof bytes before gossip amplification. Proofs are checked at admission and at finalization and are not part of the committed state, so pruning them does not change the state root. It does change what a new node can check: a node that joins from a snapshot trusts that snapshot and the validators who signed it, rather than re-checking every historical proof.

## 7 Consensus and State

### 7.1 BFT Consensus

ZCrypt is an ABCI application driven by CometBFT [21]. Blocks are final when they commit, with no reorganizations and no confirmation heuristics. The reference deployment targets a one-second block interval. The application validates a transaction at mempool admission (full proof verification, anchor and nullifier checks, no state mutation) and then again, authoritatively, when the block is finalized.

All value flows through the shielded transfer. The remaining transaction types are a closed set of transparent operations: bond, unbond, delegate and undelegate; reward and bridge-fee claims; asset registration, authority rotation and reserve-key rotation; the escrow operations of the bridge and of in-pool swaps; Bitcoin header relay and re-anchoring; and the sentinel's retirement. Anything else is rejected. There is no script layer and no general-purpose execution, so there is nothing for an attacker to program.

### 7.2 The State Root

Every block commits an application hash

$$H\big(root_{\mathrm{nf}},\ root_{\mathrm{tree}},\ root_{\mathrm{stake}},\ root_{\mathrm{assets}},\ root_{\mathrm{escrow}},\ root_{\mathrm{btc}},\ root_{\mathrm{immature}},\ root_{\mathrm{sentinel}},\ root_{\mathrm{anchors}}\big)$$

over the nullifier set, the commitment tree, the complete staking ledger (stakes, delegations, rewards, claim indices, the unbonding queue, tombstones, validator keys, per-key nonces, and total issuance), the asset registry (each asset's supply counter, cap, authority epoch and reserve-key history), the escrow book, the Bitcoin header chain and its claimed-outpoint set, the immature-mint queue, the bootstrap sentinel record, and the window of live anchor roots. Total issuance is inside the staking digest, so "supply equals the emission schedule" is a claim anyone can check against one committed hash.

Node state lives in a database written with a synchronous batch at every commit, with a full backup file written atomically every thousand blocks. A node restarted from that state reproduces the committed root exactly. A database that fails to load halts the node instead of letting it rejoin from genesis and pretend nothing happened.

### 7.3 Dynamic Validator Set

Nobody configures the validator set. It is a function of the staking ledger, with one bootstrap exception described next. Each block the application diffs the ledger's validator set before and after the block's transactions and pushes the changes to CometBFT: a bond adds a block producer at one unit of voting power per whole staked coin, an unbond or a slash removes one, and delegation changes reweight. All of it is derived from state under the state root.

### 7.4 Bootstrap Liveness Guarantor

A BFT chain cannot finalize a block until an honest supermajority of stake exists, and at genesis there is no such thing. So the genesis app_state may name one foundation validator as a *sentinel*. While it is active, the application pins its voting power at max(floor, 2·others+ 1), just over two thirds of the total, and the chain keeps producing final blocks before an independent validator set has formed. During bootstrap that validator's power has nothing to do with its stake, and the network is not decentralized. Its identity and its inflated power are both committed in the state root, so at least nothing is hidden.

The sentinel retires to its ordinary staked power once external stake crosses a threshold set in genesis, provided no single external validator key holds more than a third

of that stake and the condition has held for a dwell window of 1,000 blocks. The bound is per consensus key, so one party can meet it by splitting stake across several keys; the dwell stops a spike that does not last. The sentinel can also be retired early by a transaction signed with its own consensus key. Once it retires, the set is purely stake-weighted and no node is special.

Two limits remain. The protocol does not make the sentinel bond stake, so while it is active the evidence-triggered slashing of §8 has nothing of its own to burn; the launch runbook bonds the foundation's consensus key for that reason, but nothing enforces it. And any validator with more than two thirds of the power can decline to commit the block carrying evidence against it, at the cost of halting the chain in full view. A slashable genesis bond, retirement on tombstone, and a hard sunset height are on the list.

# 8 Monetary System and Staking

## 8.1 Emission

Maximum supply is 21,000,000 ZCRYPT with six decimals, at a one-second block time.

Twenty percent, 4.2M ZCRYPT, goes to the foundation, but not as liquid coins. It vests on an on-chain schedule that is purely a function of height: 50,000 is liquid from genesis, for validator bonds and operations; at month twelve the vested total steps to 25% of the allocation, 1,050,000, which already includes the 50,000; then 87,500 per month for 36 months, through month forty-eight, adds the remaining 3,150,000. Consensus rejects any foundation claim that runs ahead of the cumulative total.

The other 16.8M is the budget for validator and delegator rewards: 0.14 ZCRYPT per block (140,000 atomic units), halving every 60,000,000 blocks, about 1.9 years. The integer shift reaches zero at the 18th halving, about 34 years. Summing the eighteen non-zero eras mints exactly 16,799,640 ZCRYPT, 360 less than 16.8M. With the foundation's 4.2M the scheduled issuance before burns is 20,999,640 ZCRYPT; the remaining 360 are never minted. A test pins these totals. Rewards pay out every 100-block epoch. There is no mint authority anywhere in this path.

The 0.01 gas burn and slashing burns destroy supply permanently, so net supply ends strictly below 21M.

## 8.2 Bonding as Provable Burning

A validator joins by spending shielded coins into the burn key. The bond transaction has an ed25519-signed header (the consensus key that will sign blocks, the stake, a reward address, the validator's commission, and a per-key monotonic nonce, all signed with the chain identifier so the authorization works once and only on this chain) plus a shielded spend whose first output must be the revealed-value commitment of exactly the stake, bound to $pk_{\text{burn}}$. The pledge spend also burns the 0.01 gas. The stake provably leaves the pool.

The pledge proof is tied to that header: consensus key, stake, reward address, commission and nonce form an envelope that goes into the proof's auxiliary digest, so someone who sees a pledge in the mempool cannot wrap it in their own header and claim it. Delegations bind the delegator identity, target and reward address the same way. The stake amount is public, as it has to be, but where the coins came from is not. Minimum self-bond is 10,000 ZCRYPT. Delegation works the same way, for any positive amount, to any bonded validator.

## 8.3 Rewards, Unbonding, Slashing

Epoch rewards split by weight $w_v = (\text{stake}_v + \text{delegated}_v) \times \text{blocks signed}_v$, so an absent validator earns nothing no matter how much it staked. Each validator sets its own commission on the delegated share at bond time, in basis points, with a default of 10%, and the rest divides pro rata. Rewards accrue to a ledger keyed by reward address and are claimed with a gasless transaction that mints coins straight into the pool at commitments the claimant's wallet can derive. The claim names the amount and the key's current claim index; the index advances with every minted coin, so a stale claim is refused and two claims of the same amount are different transactions.

Unbonding removes the validator, cascades its delegations, and parks the stake in a queue for 10 days of block time. When it releases, the stake is a claimable balance that comes back into the pool through the same claim path. Equivocation or light-client-attack evidence from CometBFT burns 5% of the offender's own stake and 5% of each delegation, force-unbonds the remaining 95% of both through the queue, and tombstones the validator's identity for good. Delegators lose that same 5% and cannot jump ahead of it, because undelegation goes through the same queue.

A client that has been offline longer than the unbonding period cannot tell a long-range rewrite, built from keys that have since unbonded, from the chain the current validator set extends. The client ships a checkpoint for that distinction: a height, the application hash, and the hash of the bonded validator set. The compiled checkpoint is the empty node at height 0, the state before genesis mints anything. At the checkpoint height the state root and the validator-set hash must match, the match is recorded, and a snapshot above the checkpoint loads only with that record. A launch replaces the compiled checkpoint with one inside the 10-day window, and a client offline longer than that window needs a new checkpoint from a source it trusts.

# 9 A Multi-Asset Shielded Pool

The pool does not care what asset a coin is. The asset identifier inside each commitment is one BabyBear element, the reduced form of a ticker hash. The circuit enforces that a transaction is single-asset and conserves value, but the identifier never appears on the wire. Every registered asset therefore shares one anonymity set; ZCRYPT transfers, which use the native gas form, are distinguishable from them (§5.7).

Assets are created in a first-come registry. A ticker of 2 to 10 alphanumerics starting with a letter, case-insensitive, maps by hash to a field element, with a declared decimal precision and a maximum supply the registry never raises afterwards. Registration rejects a ticker whose reduced identifier collides with one already registered, including zBTC and the native identifier 0, so a new ticker cannot be enrolled as an alias of an existing asset. The reduced space is about 31 bits; birthday collisions make a fresh ticker un-registerable once tens of thousands of assets exist, but never make two registered assets share a lane.

Minting only happens through the escrow mechanism of §10, authorized either by the asset's registered threshold authority or, for zBTC, by an in-consensus Bitcoin SPV proof, and consensus caps it at the registered maximum. The per-asset supply counter is in the state root, and redemption of a bridged asset decrements it. The native asset is never registered and has no such counter: it never leaves the system, so nothing outside the pool can be reconciled against it (§14). The zBTC ticker is special-cased: consensus refuses any registration of it that does not carry exactly eight decimals, the 21-million cap and the SPV mint form, so nobody gets to choose the bridge asset's parameters.

## 9.1 Threshold Mint Authority

A mint authority is an $M$-of-$N$ threshold of ML-DSA-65 keys [22], the lattice signature standard that goes with the ML-KEM encryption of §3.3, so issuance is post-quantum end to end. The registry stores only a 32-byte commitment: a hash of the threshold and the sorted key set of up to 32 keys. To open a mint escrow the submitter presents the key set and at least $M$ signatures over a message that binds the chain identifier and the exact escrow parameters (asset, value, hashlock, timeout, recipient key, the settlement commitment, and the hash of the note that carries the blind) under a protocol context string. The VM checks that the key set hashes to the registered commitment and that the threshold verifies. A signature therefore authorizes one specific issuance and nothing else, and replaying it collides with the deterministic escrow identifier.

Authorities rotate on chain. Each asset carries a monotonic authority epoch, committed in the state root. A rotation presents the current key set with $M$ signatures over (chain, asset, current epoch, new commitment); on acceptance the registry swaps the commitment and bumps the epoch, so a rotation signature works once and an old key set keeps no power. Signers never put a secret on chain.

A single user issuing a personal asset uses a 1-of-1 authority derived from their wallet seed; a consortium uses a real multi-party set. zBTC is the one exception: it has no threshold authority at all and is gated by the SPV proof of §10, so no group of signers can mint it.

# 10 Bitcoin Interoperability

Bitcoin enters the pool as zBTC, a registered shielded asset with eight decimals and a 21M cap. Each satoshi locked on Bitcoin backs one base unit. At maturity the depositor receives that amount minus 10 basis points, and the cut accrues to ZCRYPT stakers in zBTC, by the same weight as the block reward. The fee is taken in the bridged asset because there is no price oracle; the full lock stays backed one for one. There is no discretionary mint authority. Every validator verifies Bitcoin itself, through an SPV client inside consensus, and mints only against a proof that real BTC was locked.

## 10.1 An In-Consensus Bitcoin Light Client

The chain keeps a Bitcoin block-header chain as part of its own state, digest included in the state root. Anyone can submit a batch of 80-byte headers in a relay transaction; the relayer is untrusted because each header carries its own proof of work. Validators check that a header links to its stored parent, meets the target consensus requires at its height, follows the retargeting schedule (Bitcoin's 2015-interval timespan with the fourfold clamp, off-by-one and all), and post-dates its median-time-past, and they follow the heaviest-work chain the way Bitcoin does. Mempool admission runs the same validation as a dry run, so a header that could not enter the chain cannot occupy the bounded per-block relay budget, which is only charged for batches consensus actually accepts. A production deployment pins a recent header as a checkpoint. Rewriting this chain would mean out-mining Bitcoin, and one honest relayer is enough to defeat one that withholds.

## 10.2 The SPV Mint

zBTC's registered authority is not a key-set commitment but a commitment to one Bitcoin reserve public key. To mint, a party submits an inclusion proof: the raw funding transaction plus a Merkle branch showing it sits in a block on the chain's best header chain. Consensus accepts it if the block is buried to a network-dependent minimum depth (six blocks on mainnet); if the branch reconstructs the block's Merkle root, the same proof carries the block's coinbase at index 0, and neither the funding transaction nor the coinbase serializes to 64 bytes, which closes the ambiguity between a transaction and an

inner node in Bitcoin's Merkle tree; if the funded output pays the pay-to-witness-script-hash of exactly the swap's HTLC, with the shared hashlock, the pinned reserve key as claimant, the depositor's refund key and a safe refund locktime; and if the output holds at least the satoshis being minted, and at least the network's minimum deposit.

The depositor's spendable coin is the minted amount minus the 10-basis-point fee. Binding the deposit to the reserve key stops the depositor from taking the success path: a deposit locked to a key the depositor also controls fails the script check. Each funding outpoint is recorded in a consensus set the first time it mints, so one deposit mints once.

The open and the preimage-revealing claim both require the consensus header tip to be fresh. The refund-lock margin is measured against that tip, so if its timestamp is more than four hours behind the block clock the mint is refused until headers catch up; otherwise a stalled relay would let a depositor refund on Bitcoin and still claim. A claim is also refused once the tip plus the reserve's sweep margin, 288 Bitcoin blocks on mainnet, reaches the refund lock, so at the moment of the claim the reserve has that window left to sweep.

No signature says whether a mint exists, how big it is, or what backs it; Bitcoin's proof of work decides all of that. The one signature in the flow chooses the recipient. Whoever opens the mint must present an ECDSA signature by the deposit's Bitcoin refund key over the chain identifier, the funding outpoint, the chosen shielded recipient key and the escrow parameters, so only the depositor, who set that refund key in the SPV-proven script, can say where the zBTC goes. That shuts a front-running attack in which a third party sees the public deposit proof and opens the mint to themselves. The signature is secp256k1 and ZCrypt consensus verifies it; Bitcoin never sees it. A quantum forgery of the refund key redirects the mint to a recipient the depositor did not choose.

Reserve custody is a keyholder assumption, and it should be read as one. Consensus constrains the mint proof, so zBTC is not created without an SPV proof that a funding output paid the pinned key. It does not constrain what that key does afterwards. The accumulated BTC is spendable by that key alone, and losing it leaves the zBTC supply intact while stranding redemption. The reserve key can be rotated on chain by a transaction signed with the current key over the chain identifier, the asset, a monotonic epoch, and both keys. Deposits already mined to a retired key, plus a short grace of six Bitcoin blocks, still mint, so nothing in flight is stranded, and every later deposit has to pay the new key, which wallets read from the registry. Rotation only ever moves between reserve keys, never to a signer set, so zBTC stays SPV-gated. Because consensus only checks that deposits pay the pinned public key, that key can itself be held by threshold ECDSA among independent operators without any protocol change. How it is held is a deployment decision, and a deployment should say what it is.

## 10.3 Maturity and Reorg Safety

An SPV proof shows a deposit was confirmed, but Bitcoin only offers probabilistic finality. A minted zBTC coin enters the shielded pool and becomes unlinkable, so there is no clawing it back; the peg has to stop an unbacked coin from ever being spent. The design borrows Bitcoin's coinbase-maturity rule. A newly minted coin is held immature, outside the spendable pool and therefore outside the anonymity set, until its deposit is buried past reorganization risk. Every block, consensus sweeps the immature set.

The required depth scales with value. Orphaning a deposit costs, at minimum, its depth times the block reward in forgone honest mining, so the attack only pays when the unbacked value exceeds that. The depth therefore grows with the minted value, floored at the mint's minimum confirmations and capped at 100 blocks on mainnet, near Bitcoin's coinbase maturity. Small transfers mature in about an hour; large ones wait deep, which is where an attacker would bother.

Orphaning is not an immediate void, because a benign reorganization usually re-mines the same deposit into a different block. Consensus opens a grace window during which a relayer may re-anchor the immature mint with a fresh inclusion proof for the same deposit on the new best chain; the anchor moves forward and maturation resumes. A deposit that never reappears is voided when the grace runs out, and the supply counter is burned back. All of this is a function of the header chain in the state root, so validators agree on which coins mature, re-anchor, or void.

## 10.4 The Atomic Swap

Minting and redemption are two hash-time-locked contracts in the classic atomic-swap shape [23], sharing one untagged SHA-256 hashlock so that a single preimage satisfies Bitcoin script's OP_SHA256 and ZCrypt's escrow check alike. On Bitcoin the contract is a standard P2WSH output: preimage plus signature to the claimant, or a signature alone to the refunder after an absolute OP_CHECKLOCKTIMEVERIFY height. The script pins the preimage at 32 bytes with OP_SIZE, matching ZCrypt's rule. On ZCrypt a consensus escrow locks a public (value, asset) pair under a hashlock and a block-height timeout. The beneficiary chooses the blind of the settlement coin; an opening proof shows the commitment opens to the public value and asset and the beneficiary's key, and the blind travels only in the encrypted note. The swap amount is public, as an atomic swap needs, but the settled coins go back into the private pool. Consensus reserves an open escrow's settlement commitments until it settles, so nobody can pre-insert the leaf and make the settlement collide, and an escrow

whose timeout is reached is expired by the node at the end of that block, refunding its locker without a further transaction.

Going from BTC to zBTC, the depositor picks a 32-byte random preimage, funds the Bitcoin HTLC (claimable by the reserve with the preimage, refundable to the depositor after its locktime), waits for confirmation, relays the intervening headers, and submits the mint proof. Consensus opens the mint escrow, and when the depositor claims with the preimage it mints immature zBTC. The reserve reads the revealed preimage off ZCrypt and sweeps the BTC. If the proof is never submitted the depositor refunds on Bitcoin; if the claim never comes the escrow expires and nothing is minted. Maturity checks that the funding transaction is still included on the best header chain. It does not check that the reserve's sweep has confirmed: if the reserve is offline past the refund lock, the depositor refunds the BTC and the coin still matures. The sweep margin buys the reserve a window, and solvency of the peg depends on the reserve using it. The operator is told to wait until the deposit is past maturity plus the reorg grace before sweeping, so a voided mint does not strand BTC already taken.

Going from zBTC to BTC is the same shape with the roles swapped. The redeemer locks zBTC in a burn escrow, paying 0.01 ZCRYPT of gas through the bundled gas note of §5.7 and burning none of the zBTC as a fee. A liquidity provider funds a Bitcoin HTLC claimable by the redeemer, the redeemer's Bitcoin claim reveals the preimage, and the provider uses it to finalize the burn and reduce tracked supply. Every failure path refunds the escrow. The chain burns the zBTC and does not pay the provider; whatever reimburses the provider is an arrangement off the protocol. In the reference deployment the provider is the holder of the single reserve key.

The construction removes a federation that must sign each mint: issuance needs an SPV proof of a reserve-bound funding output, and the protocol-pinned cap bounds supply. It keeps one secp256k1 key that can spend the locked coins, a maturity delay, and a dependence on that key being online for both the sweep and the redemption.

## 11 Private In-Pool Swaps

The same escrow machinery lets two parties trade one shielded asset for another without either leaving the pool: peer to peer, no order book, no pooled liquidity. A timed-out lock returns to its locker, so a stalled counterparty does not take the principal.

The swap escrow locks value out of the pool the way a redemption does, with a shielded spend whose first output is a revealed-value commitment to the burn key, alongside the asset, a hashlock, a timeout, a counterparty key and a refund key. Claiming with the preimage before the timeout re-materializes the value as a pool coin for the counterparty; expiry re-materializes it for the locker. Supply counters do not move. A swap changes ownership, not issuance, and needs no authority.

Two parties agree a price off chain; the chain never sees or checks the ratio. The taker picks a random preimage and locks their side, say $x$ of asset $A$, claimable by the maker. The maker sees the hashlock on chain and locks $y$ of asset $B$ under the same hashlock, claimable by the taker. The maker's lock carries the shorter timeout and the taker's lock expires two hours of blocks later, since the taker reveals the preimage and the maker must still have time to claim after that reveal; a maker wallet refuses any quote that does not respect this order. The taker claims the maker's escrow, revealing the preimage, and the maker claims the taker's with it. If either side stalls, both escrows expire back to their lockers. It is the Bitcoin swap of §10 with both legs on one chain.

When one leg is ZCRYPT the order is fixed so that the other side needs no ZCRYPT at all. The ZCRYPT giver locks first, whichever role it holds, and its lock is a native lock: the 0.01 fee is burned inside the lock's own proof, and it burns a second 0.01 to sponsor the counter-lock, which it names by escrow identifier (the identifier is a function of the terms, so it can be named before the lock exists). The other side then locks its asset with no gas proof. Consensus admits that lock only if the ZCRYPT lock it points at is open, is the other leg of this swap (same hashlock, each party's claim key the other's refund key), named exactly this lock, and has not sponsored one before. The sponsored lock expires a fixed margin after its sponsor. A sponsorship pays for one named lock and nothing else, so it cannot be captured by a stranger, and a sponsor whose counterparty never appears is out 0.02 ZCRYPT and gets its principal back at expiry. The newcomer's first ZCRYPT coin, claimed from that escrow, then spends on its own.

Privacy composes the same way it does for the bridge. The amounts and the hashlock are public, because an atomic swap needs them, but which coins funded each lock hides behind the whole-pool membership proof, change comes back as an ordinary commitment with an encrypted note, and the coins each side receives are standard pool coins. A swap lock burns 0.01 ZCRYPT in its own proof for a ZCRYPT leg, by its sponsor for a sponsored leg, or in a separate gas note when neither leg is ZCRYPT; the locked asset is never charged.

## 12 The Wallet

The reference wallet ships for desktop, iOS and Android with the full prover embedded. No third party ever sees a witness.

The wallet keeps an incrementally synced copy of the commitment-tree leaves, rebuilds the tree locally, and derives Merkle paths for its own coins. A payment selects up to two coins. For ZCRYPT it builds one native

transaction (burn, payment and change in a single proof, about 1.2 s on a laptop); for any other asset it builds the main transaction and a gas note and proves both, about 2.5 s. Either way it runs the auditability VM as a last cross-check before broadcasting. A wallet whose whole balance is one ZCRYPT coin is never stuck, because the gas comes out of that coin. When two coins cannot cover a payment the wallet reshapes its own coins first, with the same proofs, rather than asking the user to. Change always goes to a fresh stealth sub-address, and every output slot carries a fixed-size note, real or cover, so the delivery mode cannot be fingerprinted.

Scanning fetches only the blocks that contain transactions and trial-decrypts every note against the account key, the stealth change accounts (with a gap limit of twenty), and every stealth seed the wallet has generated. A stealth payment arrives as an out-of-band zct1 note; the chain carries a cover note in its place. A bearer payment arrives as the file of §4.3, and opening it sweeps the coin and burns the gas coin from the file.

Every payment the wallet sends, other than a reshape of its own coins, writes a local receipt. The file begins zcrypt-receipt/1 and records the kind (normal, stealth or bearer), the amount, the asset, the commitment and the blind. A normal or stealth receipt names the recipient address and the owner key that address commits to; a bearer receipt names no recipient and records the nullifier, not the spend secret; a stealth receipt also carries the out-of-band note. Anyone holding the file recomputes the commitment and looks for that leaf, which is how a disputed payment is checked without moving the coin. The receipt sits in the encrypted wallet, and a restore from the mnemonic does not bring it back.

Asking a node "is nullifier $X$ spent?" would tell it which coins are yours, so the reference wallet never asks. It downloads every transaction-bearing block, proofs included, and matches the published nullifiers against those of its own coins locally. All the node learns is that someone did a full-block sync. Nodes also serve authenticated inclusion and non-inclusion proofs against the registry root for anyone who prefers a point query. Trusting such a proof completely would need the registry root tied to the block's application hash through validator-signed headers, a light-client step the reference wallet does not yet perform.

Node RPC can go over Tor. It is off by default, so a fresh install reaches the public node without a Tor daemon, and when it is on it fails closed if misconfigured. The mobile wallets have no Tor. The mnemonic is encrypted at rest under an Argon2id-derived key [24] (64 MiB, 3 passes), and the coin database under a key derived from the seed. The wallet exposes the whole product: balances per asset, the three payment instruments, view-key export, reward claims, asset registration and minting, and both directions of the bridge, with automatic recovery of swaps that were interrupted partway. Every bridge operation uses a fresh shield key, so a user's bridge operations are not linked to each other by key reuse.

## 13 Privacy Analysis

Someone watching the chain sees the following, action by action.

A transfer shows two nullifiers, three commitments, three uniform ciphertexts, the 0.01 ZCRYPT gas burn, an anchor root, a proof, and a tag that says whether the transfer moved ZCRYPT or some other asset (§5.7). It shows no amounts, no participants, no arity, no link from the nullifiers back to any earlier commitment, and within the non-ZCRYPT class no asset type. A receipt is a voluntary exception: publishing it reveals the amount, the asset, the blind and, for a normal or stealth payment, the recipient. The spend secret is never in a receipt.

A reward claim shows the amount and the recipient shield key. Rewards are public income, so this is intended. The minted coins use deterministic blinds derived from public values, which makes them identifiable when they are born; they become anonymous once spent.

A bond, delegation or unbond shows the stake and the validator identity, public on purpose, with the source coins anonymous.

A bridge or in-pool swap shows the amounts, the escrow parameters, and the claimant's shield key. A bridge deposit also publishes the raw Bitcoin funding transaction, which ties the depositor's Bitcoin history to that shield key. The settlement commitment is in the open transaction, so the coin is identifiable at birth and anonymous once spent. A user who reuses one shield key across bridge operations links them together, which is why the reference wallet derives a fresh key per operation. The coins that funded each leg stay anonymous.

An asset registration or authority rotation shows the ticker, decimals, cap, authority commitment and epoch, and never a signer's secret.

The metadata that leaks is small for transfers and larger at the edges, where an amount has to be public for consensus to enforce a rule about it. Timing and network-origin metadata are addressed by optional Tor, a uniform transaction shape, cover notes, and the absence of any transparent value layer that would sort users into pools with different levels of privacy. The anonymity set follows directly from the membership proof: because membership is proven against the whole commitment tree, every spend hides among every coin in history, spent or not. There is no decoy-selection distribution to skew and no ring-intersection attack to run.

## 14 Security Considerations

Value integrity rests on the STARK, on the hash, and on consensus being correct. The STARK's conjectured list-decoding figure is 244 bits and the Johnson bound the node checks is 132 bits (§3.2); the Johnson figure has no

quantum analysis behind it. Poseidon2 preimage resistance backs spend authority and the burn key. Commitment binding is the compression of §3.1. A soundness failure that creates a coin inside the pool is not visible in the issuance digest: that digest proves the foundation vesting and the epoch rewards followed the schedule, not that the pool contains only those coins. zBTC has a partial check of this kind, since its supply counter rises on mint and falls when a redemption finalizes. The native coin never exits and has no such counter.

A field element has one canonical byte serialization anywhere it matters: eight little-endian bytes holding a value below the field order. Transaction parsing rejects non-canonical limbs, which rules out the double-spend that respends a coin under a second encoding of the same nullifier.

Against replay, a proof binds a chain-domain tag, so it cannot be replayed on another chain; an anchor from a bounded window, so it cannot be built against ancient state; a transaction-binding hash over its ciphertexts, commitments, fee and proof bytes, stored as the nullifier leaf; and, for stake pledges and escrow locks, the plaintext parameters around it, so the proof cannot be re-wrapped under someone else's header. The gas proof of a bundled transfer, a swap lock, a registration, an escrow open or a redemption commits to the transaction it pays for, and a native transfer's burn is an output of the proof that moves the value, so there is no gas to detach. A sponsored swap lock is admitted against one named sponsor, once. Phantom nullifiers are derived from each real input's secret coin tag, so they never collide across transactions. A repeated commitment is rejected at the tree.

Safety violations in consensus are punished by slashing 5% of the offender's own stake and of each delegation, force-unbonding the remaining 95%, and permanent tombstoning. Delegators share the 5% and cannot exit ahead of it. Once the sentinel has retired, voting power comes only from stake that left the pool. Unbonding returns that stake after 10 days, so holding power costs the lock and, on equivocation, the slash; it does not destroy the stake. While the sentinel is active, the foundation node holds a disclosed, state-root-committed supermajority, and that is the trust assumption of the bootstrap phase.

Mint authorizations are ML-DSA-65 threshold signatures over the complete escrow parameters, so one authorization fits one issuance, cannot double as a rotation, and cannot be replayed. Rotations are bound to a per-asset epoch in the state root. A malformed authorization bundle (out-of-range or duplicate signer indices, undecodable keys or signatures, trailing bytes) is rejected whole.

On the quantum question: spends use no discrete-logarithm assumption. Proofs and spend authority are hash-based, note confidentiality is lattice-based, and recorded ciphertexts are safe against harvest-now-decrypt-later. The classical cryptography that remains is not all peripheral. ed25519 on validator consensus keys is an availability matter. secp256k1 on the Bitcoin script is Bitcoin's signature. The secp256k1 signatures that choose a zBTC mint's recipient and rotate the reserve key are checked by ZCrypt consensus, and a quantum forgery of those keys redirects mints or rotates the reserve inside ZCrypt's own rules.

The circuit is differentially tested against an independent wide-trace implementation and against the transparent VM on sampled witnesses; wallets re-run the VM before broadcasting; nodes reproduce their state root from persisted state and refuse to start if they cannot. The check consensus applies to a transfer is the AIR. The seven-opcode program is the specification and the prover-side cross-check, not the acceptance path. Zero knowledge of published proofs is the unproved hiding layer of §3.2.

## 14.1 Machine-Checked Verification

A companion development lifts part of the testing to machine-checked proofs in Lean 4 [30], kernel-verified with no sorry. Three things are established.

First, the transfer's value-integrity specification (the VM of §6) and the arithmetic at the core of the circuit are proven directly: an accepted transfer creates no value, and the multi-limb range-checked carry provably cannot wrap, so conservation in the field is conservation over the integers. The coin-layer results are reductions: if two openings give the same commitment, or a commitment equals a nullifier or a public key, the proof returns the two hash inputs that collide. Collision resistance of the compression is the assumption, and the file does not discharge it.

Second, so that the proofs are about the source rather than a hand transcription, the value-integrity core of the VM is mechanically extracted from its Rust into Lean with Charon and Aeneas [31], the pipeline the Ethereum Foundation's zkEVM verification project has applied to Plonky3 primitives [32], and the no-inflation theorem is proved again against that generated model.

Third, the production AIR itself is modeled constraint by constraint. The tall circuit's forty assertion sites, with their row gating, lane layout, public-input offsets, 128-row geometry, two inputs, three outputs and 62-level paths, are transcribed into Lean as a predicate on traces, and a soundness theorem shows that every satisfying trace yields the proven statement of §5: 26-bit limbs, conservation over the integers, one shared asset, ownership of each input by its spend secret, a canonical nullifier that is a function of the opening alone, zero-valued dummies, and Merkle membership of every active input under the public root. The Poseidon2 permutation is an arbitrary function in that model; nothing about it is assumed, not even injectivity. The BabyBear modulus is proved prime inside Lean. The lane-level finding of the October review, where the inner nullifier block absorbed

a sender-chosen blind, is in the model as a theorem. A companion file pins the FRI parameters and proves the arithmetic of the 132 and 244 figures, and a crate test fails if the Lean constants drift from the compiled ones.

What is verified unconditionally is the transfer specification's arithmetic and the constraint system's wiring, for any hash. Outside the model sit the Poseidon2 round constraints, the FRI soundness of §3.2 (the parameter arithmetic is checked, the proximity-gap theorem is not), the lattice assumptions of §3.3, the prover and verifier code, and the chain-domain and notes-digest public inputs, which the AIR does not constrain and the transcript binds. The AIR model is a hand transcription of the source, checked by a second reading, and any relayout of the circuit breaks the build until the model is updated. None of this replaces an external audit.

# 15 Deployment Models

The protocol described here is neutral: a shielded pool with hash-based proofs on the spend path, lattice note encryption, threshold lattice signatures on issuance, and selective disclosure, with a Bitcoin bridge that still checks secp256k1 signatures inside ZCrypt's own rules. It could be the base for private digital cash, or the technical infrastructure for a central bank digital currency. I advocate neither here.

## 15.1 Central Bank Digital Currency

CBDC design is caught between two demands. Monetary authorities need control over issuance and compliance; citizens have a legitimate expectation of transactional privacy, which is what physical cash has always given them. A recent BIS working paper [25] puts it this way: "While some technological solutions for hard privacy show significant promise, further development is necessary." The primitives here happen to fit. The threshold mint authority (§9.1) is an exclusive, consensus-enforced issuance mechanism that spreads signing power across $M$-of-$N$ institutions and can be rotated without redeploying anything. The shielded pool keeps ordinary commerce private. The nullifier registry gives the global double-spend prevention a monetary authority needs. The consensus layer can be run by the issuer or by licensed delegates, and the transfer layer stays private regardless.

A CBDC on these primitives would get privacy-preserving compliance through tiered disclosure, the inverse of designs that start from full visibility and carve out a little privacy. Routine transactions are fully private, like cash. The issuer sees aggregate nullifier volume, a privacy-preserving proxy for velocity, and not individual transactions. Where policy demands more, disclosure is additive and scoped to keys: view keys (§4.3) let a wallet show its incoming flows to a designated auditor or, under a court order, to a financial intelligence unit, without giving up the ability to spend. Privacy is the base layer, and compliance goes on top of it.

There is a surveillance risk, and it should be said out loud. Privacy infrastructure deployed by a state can be repurposed. Threshold reporting is a policy surface that can be lowered to zero, judicial disclosure depends on courts that may be under political pressure, and a mandate to hand over view keys at issuance would turn a privacy tool into an escrow system. As a matter of architecture, the proof system contains no compliance hooks, trapdoors, or key escrow. Privacy is a property of the cryptography and not a setting that can be switched off later.

Central banks work on generational timescales, which makes the quantum question urgent for them. A CBDC on elliptic-curve cryptography faces store-now-decrypt-later: capture the traffic today, decrypt it when the machines arrive. Signature-based CBDC designs also run into a quantum-safe performance wall; BIS Project Tourbillon [26] measured roughly a 5× latency penalty and a 200× throughput reduction when it moved blind-signature eCash [27] to post-quantum primitives. The proof-based design here does not have that problem: the spend path is hash-based from genesis, and a pool payment has no signature. Whether these properties are wanted in a CBDC is for monetary authorities and the publics they answer to.

## 15.2 Private Digital Cash

Deployed as an open network, ZCrypt's instruments can move without an account. A bearer payment is the file of §4.3: a value coin and the gas that spends it. Whoever holds the file can sweep it, and so can anyone who kept a copy. Handing the file over does not take it away from the giver, which is where the comparison with cash stops: until the receiver sweeps it the file is worth what their trust in the giver is worth, and the sweep is what makes it theirs. The sweep pays gas from the file and re-keys the coin, and the previous secret dies. A stealth payment is an encrypted note, and it leaves nothing on chain that a view key can open. Finality in either case is a sweep or a claim against the network. The reference network for this deployment is chain id zcrypt-1.

This is what Chaum described in 1983 [28, 29]: ecash as a privacy-preserving medium of exchange, here without his dependence on a central mint. Remittances settle in seconds over the channels people already use, with no processor taking a cut or keeping a record. The same properties serve organisations that need confidentiality: a corporate treasury gets post-quantum protection and privacy from competitors watching the chain, funds can move without signalling intent to a market, and an aid organisation can disburse without building a ledger that a hostile actor could use to find the recipients.

## 16 Conclusion

ZCrypt makes one structural bet: a ledger is most private, and most fungible, when privacy is the substrate rather than a feature. One pool holds all the value, a shielded transfer moves pool coins, and the anonymity set is the history of the system. Around that core the rules are transparent, as bytecode specifications, an auditable emission schedule, and a state root that commits to every ledger, while the activity is hidden. The spend path builds on no discrete-log assumption: hash-based proofs with no ceremony, ownership by preimage with no signature on the spend, lattice-based confidentiality with no harvestable ciphertexts. The bridge still verifies secp256k1 signatures inside ZCrypt's own rules. Stake bonds, reward issuance and asset creation pass into the same pool through the gates described above, with the binding, soundness and custody limits of §14. What an observer can audit is that the published rules were followed. Who paid whom stays hidden. A payment that should not have an address can be a file, and the sweep is what settles it.

## Acknowledgments

Thanks to Vasily Pozdyshev for his feedback on institutional adoption, Jameson Lopp for his feedback on non-inclusion proofs, and the zero-knowledge and post-quantum cryptography communities for advancing the primitives used here.

## References

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.

[2] N. van Saberhagen, "CryptoNote v2.0," October 2013.

[3] D. Hopwood, S. Bowe, T. Hornby, N. Wilcox, "Zcash Protocol Specification," 2016.

[4] D. Hopwood, S. Bowe, T. Hornby, N. Wilcox, "Zcash Protocol Specification," version 2024.5.1 (September 2024) and later, Orchard action statement and Merkle path validity, https://zips.z.cash/protocol/protocol.pdf (serves the current build).

[5] L. Parker, "Full-Chain Membership Proofs Development," Monero Project blog, April 2024, https://www.getmonero.org/2024/04/27/fcmps.html.

[6] A. Szepieniec and T. Værge, "Neptune White Paper," April 2021, https://neptune.cash/whitepaper. The live network's generation addresses use lattice-based public-key encryption (https://docs.neptune.cash/neptune-core/keys.html).

[7] S. Bowe, J. Grigg, D. Hopwood, "Recursive Proof Composition without a Trusted Setup," IACR ePrint 2019/1021.

[8] Penumbra Labs, "The Penumbra Protocol," protocol specification, https://protocol.penumbra.zone, accessed September 2026.

[9] A. Sun Yin, "Introducing Namada: Interchain Asset-agnostic Privacy," Namada blog, May 2022, since retitled "Multichain Asset-Agnostic Data Protection," https://namada.net/blog/introducing-namada-multichain-asset-agnostic-data-protection.

[10] L. Grassi, D. Khovratovich, M. Schofnegger, "Poseidon2: A Faster Version of the Poseidon Hash Function," AFRICACRYPT 2023.

[11] National Institute of Standards and Technology, "FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," 2015.

[12] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev, "Scalable, Transparent, and Post-Quantum Secure Computational Integrity," IACR ePrint 2018/046.

[13] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev, "Fast Reed–Solomon Interactive Oracle Proofs of Proximity," ICALP 2018.

[14] E. Crites, A. Stewart, "On Reed–Solomon Proximity Gaps Conjectures," IACR ePrint 2025/2046, 2025.

[15] Polygon Zero and contributors, "Plonky3: A Toolkit for Polynomial IOPs," https://github.com/Plonky3/Plonky3, 2024; release 0.6.1, June 2026.

[16] National Institute of Standards and Technology, "FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard," 2024.

[17] H. Krawczyk, P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, 2010.

[18] Y. Nir, A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC 8439, 2018.

[19] M. Palatinus, P. Rusnak, A. Voisine, S. Bowe, "BIP-39: Mnemonic Code for Generating Deterministic Keys," 2013.

[20] P. Wuille, "BIP-350: Bech32m Format for v1+ Witness Addresses," 2020.

[21] E. Buchman, J. Kwon, Z. Milosevic, "The Latest Gossip on BFT Consensus," arXiv:1807.04938; CometBFT, the maintained implementation.

[22] National Institute of Standards and Technology, "FIPS 204: Module-Lattice-Based Digital Signature Standard," 2024.

[23] T. Nolan, "Alt Chains and Atomic Transfers," bitcointalk.org, https://bitcointalk.org/index.php?topic=193281.0, May 2013.

[24] A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson, "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications," RFC 9106, 2021.

[25] R. Auer, R. Böhme, J. Clark, D. Demirag, "Privacy-Enhancing Technologies for Digital Payments: Mapping the Landscape," BIS Working Papers No. 1242, January 2025.

[26] BIS Innovation Hub Swiss Centre, "Project Tourbillon: Exploring Privacy, Security and Scalability for CBDCs," Bank for International Settlements, November 2023.

[27] D. Chaum, T. Moser, "eCash 2.0: Inalienably Private and Quantum-Resistant to Counterfeiting," 2022.

[28] D. Chaum, "Blind Signatures for Untraceable Payments," Advances in Cryptology: CRYPTO '82, 1983.

[29] D. Chaum, A. Fiat, M. Naor, "Untraceable Electronic Cash," Advances in Cryptology: CRYPTO '88, 1990.

[30] L. de Moura, S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," CADE 28, 2021.

[31] S. Ho, J. Protzenko, "Aeneas: Rust Verification by Functional Translation," Proc. ACM Program. Lang. (ICFP), 2022.

[32] N. Klaus, J. Conejero, P. Tolmach, "A Rust-to-Lean Verification Pipeline with AI Provers: An Experience Report," arXiv:2605.30106, 2026.